Lab session 6: Programming Fundamentals

Lab session 6 consist of 10 exercises. Each exercise is worth 1 grade point. All exercises are Dafny exercises.

Exercise 1 is a Dafny programming exercise (like we did before in C), and does not require any program verification. The purpose of this exercise is to get comfortable with the compiler and a small IO (input/output) library that we made for Dafny. You include it by placing the line `include "io.dfy"` (which can be downloaded from Themis) at the top of your program. You should copy the file `io.dfy` into the directory in which you are making your lab exercises. Note that you should only submit your program file to Themis. Do not submit the file `io.dfy` to Themis (testing will fail if you do this).

The remaining exercises are problems that involve correctness verification. These problems consist of only a single method of which correctness needs to be proven. Therefore, these problems do not have a `Main` method, nor do they need to include `io.dfy`.

The following program demonstrates how to use the IO library to read an `int` and a `nat` from the input. It outputs the sum of the two numbers. You can download the file `io.dfy` from Themis.

```
include "io.dfy"

method Main()
{
    var a : int, b : nat;
    a := IO.ReadInt();
    b := IO.ReadNat();
    print "a+b=", a + b, "\n";
}
```

Problem 1: Missing number

The input for this problem consists of 9 decimal digits, without duplicates. The output should be the missing digit that is not in the input.

Example 1:

input:
```
0 1 2 3 4 6 7 8 9
```
output:
```
5
```

Example 2:

input:
```
1 3 6 4 2 7 9 8 0
```
output:
```
5
```

Example 3:

input:
```
1 3 4 5 9 8 6 0 7
```
output:
```
2
```

Problem 2: Dafny specification 1

From Themis you can download the file `exercise2.dfy`. It contains the following method.

```
method exercise2(x: int, ghost X: int) returns (y: int)
requires 7*X - 3 < x < 7*X + 5
ensures y == X
{
   // implement the body of this method
}
```

You are required to implement the body of the method such that the Dafny verifier accepts it.

Problem 3: Dafny specification 2

From Themis you can download the file `exercise3.dfy`. It contains the following method.

```
method exercise3(x: int) returns (y: int)
ensures y>=1 && (y==x  || y == 1 - x)
{
   // implement the body of this method
}
```

You are required to implement the body of the method such that the Dafny verifier accepts it.

Problem 4: Dafny specification 3

From Themis you can download the file `exercise4.dfy`. It contains the following method.

```
method exercise4(x: int) returns (y: int)
ensures y>0 && (y==2*x + 1  || y == -2*x)
{
   // implement the body of this method
}
```

You are required to implement the body of the method such that the Dafny verifier accepts it.

Problem 5: Dafny specification 4

From Themis you can download the file `exercise5.dfy`. It contains the following method.

```
method exercise5(x: int, ghost X:int) returns (y: int)
requires x>0 && (x==2*X + 7  || x == 2*X + 2)
ensures y == X
{
   // implement the body of this method

}
```

You are required to implement the body of the method such that the Dafny verifier accepts it.

Problem 6: Fill in the ??

From Themis you can download the file `exercise6.dfy`. It contains the following method.

```
method exercise6(ghost X: int, y: int) returns (z: int)
requires 2*X - y == 3 || 4*X + y == 42
ensures z == X
{
   if ??
   {
      z := ??
   } else
   {
      z := ??
   }
}
```

You are required to replace the question marks by expressions such that the Dafny verifier accepts it. You are not allowed to add any auxiliary statements.

Problem 7: Fill in the ??

Consider the following recurrence (for $n \geq 0$):

$$f(0) = 1,$$
$$f(n) = f(n-1) + 3 \cdot n^2 + 3 \cdot n + 1, \qquad n \geq 1.$$

From Themis you can download the file `exercise7.dfy`. It contains the following code fragment.

```
ghost function f(n: nat): int
{
  if n == 0 then 1
  else f(n-1) + 3*n*n + 3*n + 1
}


ghost predicate toProve(n: nat)
{
  f(n) == ??
}

method InductionProof(n: nat)
ensures toProve(n)
{
  var i := 0;
  while i < n
  invariant 0<=i<=n && ??
  {
    i := i + 1;
  }
}
```

Find a closed expression for the function `f(n)` and replace the question marks such that the Dafny verifier accepts it. You are not allowed to add any auxiliary statements.

Problem 8: Fill in the ??

Consider the following second-order recurrence (for $n \geq 0$):

$$\begin{aligned} f(0) &= 0, \\ f(1) &= 1, \\ f(n) &= 4 \cdot (f(n-1) - \cdot f(n-2)), \qquad n \geq 2. \end{aligned}$$

In this exercise, we will use Dafny to prove that $f(n) = n \cdot 2^{n-1}$ for all integers $n \geq 1$.

From Themis you can download the file `exercise8.dfy`. It contains the following code fragment.

```
ghost function f(n: nat): int
{
  if n <= 1 then n
  else 4*(f(n-1) - f(n-2))
}

ghost function Pow(x: nat, y: nat): nat
{
  if y == 0 then 1
  else x*Pow(x, y-1)
}

ghost predicate toProve(n: nat)
{
  ??
}

method InductionProof(n: nat)
requires n >= 1
ensures toProve(n)
{
  var i := 1;
  while i < n
  invariant 1<=i<=n && ??
  {
    i := i + 1;
  }
}
```

You are required to replace the question marks by expressions such that the Dafny verifier accepts it. You are not allowed to add any auxiliary statements.

Problem 9: Integer logarithm

From Themis you can download the file `exercise9.dfy`. It contains the following code fragment.

```
ghost function exp(n: nat, e: nat): nat
{
  if e == 0 then 1 else n * exp(n, e - 1)
}

method intLog(n: nat, b: nat) returns (k: nat)
  requires n > 0 && b > 1
  ensures exp(b, k) <= n < exp(b, k+1)
{
  // implement yourself
}
```

You are required to implement the method `intLog` using a while loop. You need to decorate it with an invariant (and possibly `assert`ions) such that the Dafny verifier accepts it.

Problem 10: Recurrence to iteration

From Themis you can download the file `exercise10.dfy`. It contains the following code fragment.

```
function f(n: nat): nat {
  if n == 0 then 1
  else if n%2 == 0 then 1 + 2*f(n/2)
       else 2*f(n/2)
}

method computeF(m: nat) returns (a: nat)
ensures  a == f(m)
{
  // implement yourself
}
```

You are required to implement the method `computeF` using a while loop. You need to decorate it with an invariant (and possibly `assert`ions) such that the Dafny verifier accepts it.